CampusHub 体系结构设计文档

1. 架构概览

1.1 架构风格选择：模块化单体架构 (Modular Monolith)

本项目决定采用**模块化单体架构（多模块打包）**，并结合前后端分离与传统三层架构（MVC）的规范 。

- **不选择微服务的理由**：对于4人学生团队和10周的开发周期（约240人时），微服务的学习成本极高，服务注册、网关、分布式事务等组件属于过度设计，容易导致项目延期甚至崩盘 。
- **选择模块化单体的理由**：该架构前期能够像传统单体一样快速开发与上线，极大地降低了部署与联调的复杂度 。同时，在代码结构上严格按照业务领域进行垂直拆分，为后续可能的微服务演进打好基础 。

1.2 架构图

以下为 CampusHub 的系统逻辑分层与模块架构图（见下页）：

```
graph TD
    subgraph 表现层 View / UI
        A\[前端页面 Vue 3 / 小程序\]
    end

    subgraph 控制层 Controller / API
        B\[RESTful API 接口 / 参数校验\]
    end

    subgraph 业务逻辑层 Service \- 核心领域模块
        C1\[用户中心模块 user-service\]
        C2\[商品&资源模块 resource-service\]
        C3\[订单核心模块 order-service\]
        C4\[交易支付模块 trade-service\]
    end

    subgraph 支撑引擎
        E\[订单状态机引擎 事件驱动\]
        F\[Redis 分布式锁 / 缓存\]
    end

    subgraph 数据访问层 DAO / Mapper
        D1\[MyBatis-Plus 数据库操作\]
    end

    subgraph 基础设施层
        DB\[(MySQL 8.0 主从)\]
    end

    A \--\>|HTTP 请求| B
    B \--\> C1
    B \--\> C2
    B \--\> C3
    B \--\> C4

    C3 \--\> E
    C2 \--\> F
    C3 \--\> F

    C1 \--\> D1
    C2 \--\> D1
    C3 \--\> D1
    C4 \--\> D1
```

```
    D1 \--\> DB
```

2. 模块划分

系统后端不再是简单的水平大锅饭，而是严格按照核心域拆分为 4 个主要业务模块，确保足够支撑全业务且支持后期平滑扩展 。

1. **用户中心模块 (User Service)**
   - **职责**：负责账号注册与登录、身份认证、学生认证、权限管理及地址管理 。
   - **特性**：无状态设计，易于水平扩容 。
2. **商品&资源模块 (Resource Service)**
   - **职责**：负责二手商品的上下架、库存管理、图片展示与搜索 。
   - **职责**：负责快递代取等互助需求的发布与管理 。
3. **订单核心模块 (Order Service)**
   - **职责**：负责快递代取下单、二手交易下单以及订单的创建 。
   - **核心引擎设计**：内置独立的**订单状态机引擎**，采用状态模式+事件驱动管理接单、抢单、派单、取消、超时关单等复杂状态流转，严禁使用复杂的 if-else 硬编码 。
4. **交易支付模块 (Trade Service)**
   - **职责**：负责金额计算、模拟支付、退款、保证金及对账逻辑 。
   - **特性**：需要强事务与强一致性保障，因此在代码包层面进行单独隔离 。

3. 技术选型

- 为了满足开发速度快、易于分工，以及能够应对校园高峰期瞬间 QPS 冲高等特点，项目具体技术栈选型如下：

层次	选择	选择理由
前端框架	Vue 3 + Vite + Element Plus (或 Vant)	支持前后端分离的响应式开发，组件库生态成熟，能够快速构建出交互界面。
后端框架	Spring Boot 3 + MyBatis-Plus	约定优于配置，开箱即用，CRUD 代码开发极快，非常适合 4 人团队并行分工。

层次	选择	选择理由
数据库	MySQL 8.0	稳定可靠，支持强事务处理，用于存储用户、互助需求、订单、评价等核心业务数据。
中间件1（缓存与锁）	Redis	应对校园高峰期瞬间 QPS 冲高，存放高频访问的列表页缓存；并利用分布式锁控制同一订单的并发操作，有效防止恶意抢单与超卖。
中间件2（身份鉴权）	JWT (JSON Web Token)	实现无状态的身份验证，服务端无需维护 Session，完美契合前后端分离架构，且易于后续水平扩展。
接口规范与文档	RESTful API + Swagger	统一前后端的通信标准，通过代码自动生成在线接口文档，极大降低前后端联调时的沟通成本。
部署方式	Docker Compose	强制所有中间件（MySQL、Redis）容器化一键拉起，实现“环境即代码”，彻底消除团队成员因操作系统不同导致的环境冲突问题。

4. 架构对团队协作的赋能

此架构方案非常适合 4 人团队的协作分工：

- **需求负责人**：可以将用例与功能直接映射到上述 4 个核心业务模块中 。
- **架构负责人**：通过分层结构设计清晰的边界与接口原则 。
- **开发负责人**：组织 2 人负责后端 Service 与 Controller，1 人负责数据库 Mapper，1 人负责前端页面与联调 。
- **测试负责人**：可以根据分层结构，独立进行 API 接口测试、Service 单元测试和前端功能测试 。

5. ADR合集

ADR-001：采用模块化单体架构 (Modular Monolith)

- **状态**：已接受
- **背景**：团队仅 4 人，需在 10 周内交付。微服务虽然扩展性好，但涉及服务治理和分布式事务，对学生团队成本过高 。
- **决策**：放弃微服务，采用模块化单体架构，在单体应用内按业务领域（用户、订单、交易、资源）进行包隔离 。
- **理由**：降低了部署和联调复杂度，确保 240 人时内能闭环业务逻辑 。

- **后果**：
  - 正面：开发速度极快，本地联调无障碍 。
  - 风险：需严格执行代码审查，防止模块间产生非法耦合。
- **AI 辅助记录**：
  - **AI 初稿摘要**：建议使用微服务架构以保证高可用性和技术先进性 。
  - **人工修订内容**：强制修正为“模块化单体”，移除所有关于分布式服务治理的内容 。
  - **修订理由**：AI 忽略了真实的人力资源限制，盲目追求高级架构会导致项目烂尾 。

ADR-002：使用状态机模式管理订单流转

2. **状态**：已接受
3. **背景**：互助平台涉及接单、履约、确认、评价等复杂状态切换，硬编码 if-else 会导致逻辑碎片化且难以测试 。
4. **决策**：引入状态机模式（State Machine Pattern），通过事件驱动触发状态变更 。
5. **理由**：保证订单状态流转的一致性，防止资损和流程纠纷 。
6. **后果**：
   - 正面：逻辑高度内聚，状态流转图可清晰映射到代码。
   - 负面：初期编码工作量略有增加。
7. **AI 辅助记录**：
   - **AI 初稿摘要**：建议在 Service 层通过简单字段更新处理状态切换。
   - **人工修订内容**：改为“独立状态机引擎”，并要求状态变更必须发布异步事件 。
   - **修订理由**：简单的字段更新无法应对复杂的逆向流程（如取消、纠纷），需要更健壮的模式 。

ADR-003：利用 Redis 实现抢单并发控制

- **状态**：已接受
- **背景**：校园代取快递等场景存在明显的高峰流量，多用户同时抢单的时候易引发超卖或重复接单 。
- **决策**：在 Service 层引入 Redis 分布式锁，对订单接单接口进行限流与排队处理 。
- **理由**：利用缓存层的高性能（QPS 瞬间冲高）保护底层 MySQL 数据库 。
- **后果**：
  - 正面：防止恶意刷单和并发冲突，提升高峰期系统稳定性 。
  - 负面：系统增加了对 Redis 的依赖。
- **AI 辅助记录**：
  - **AI 初稿摘要**：建议使用数据库行锁 (SELECT FOR UPDATE) 处理并发。
  - **人工修订内容**：改为 Redis 分布式锁 + 接口限流策略 。
  - **修订理由**：数据库行锁在高并发下会导致连接池耗尽，不适合校园瞬间爆发的场景 。

ADR-004：基于 JWT 的无状态身份认证

- **状态**：已接受
- **背景**：项目采用前后端分离架构（Vue 3 + SpringBoot），需要一种轻量、易分工的鉴权方式 。
- **决策**：采用 JWT (JSON Web Token) 进行身份验证，客户端存储 Token，服务端不保存 Session 状态。
- **理由**：符合前后端分离的文档要求，且方便后续接入 CI 自动化测试 。
- **后果**：
  - 正面：服务端无状态，易于水平扩展。
  - 风险：Token 签发后难以撤回，需配合较短的过期时间。
- **AI 辅助记录**：
  - **AI 初稿摘要**：推荐使用传统的 Session 或 OAuth2 复杂流程。
  - **人工修订内容**：简化为“JWT + 拦截器”模式 。
  - **修订理由**：OAuth2 对于中小型课程项目过于沉重，JWT 在 10 周开发周期内更具可行性 。

ADR-005：基础设施强制容器化 (Docker Compose)

- **状态**：已接受
- **背景**：团队成员技术背景和系统环境不一，数据库版本冲突曾影响 P1 实验进度 。
- **决策**：强制所有中间件（MySQL, Redis）必须通过 Docker Compose 一键拉起 。
- **理由**：确保“环境即代码”，消除“我机器上能跑”的经典协作问题 。
- **后果**：
  - 正面：新成员克隆仓库后可分钟级搭建好开发环境 。
  - 风险：需要成员掌握基础的 Docker 操作。
- **AI 辅助记录**：
  - **AI 初稿摘要**：建议在本地安装 MySQL 8.0 社区版并手动配置。
  - **人工修订内容**：改为“强制 Docker Compose 部署”，并配置持久化挂载 。
  - **修订理由**：手动配置极易因版本偏差（如字符集、时区）导致联调失败，容器化是团队分工的最佳保障 。